Cheiguer Ely Cheikh – Hajar Elghelmani – Ismael Benchelh

January 10, 2026

1 Optimisation de Fosse Ultime (2D) — Pente 45° (Lerchs–Grossmann)

1.1 Question 1 — Données du problème (profits) et indexation des blocs

On modélise le gisement par une grille 2D de **5 lignes × 8 colonnes** (40 blocs). Chaque bloc possède un **profit** (P_i) (positif = rentable, négatif = stérile / coût).

Objectif : charger la matrice des profits et définir : - une fonction `block_id(r,c)` qui donne l'ID (1..40), - une fonction `rc_from_id(i)` qui retourne (r,c).

[1]:
```
import pandas as pd

PROFITS = [
    [-1, -1,  2,  1,  1, -1,  1, -2],
    [-2, -1,  2,  2,  3,  4, -1, -2],
    [-3, -2,  1,  2,  2, -1,  1, -1],
    [-3, -1,  1,  2,  1,  1, -1, -2],
    [-2, -3, -2,  1, -1, -2,  3, -1],
]
NROWS, NCOLS = 5, 8

def block_id(r: int, c: int) -> int:
    return r * NCOLS + c + 1

def rc_from_id(i: int):
    i0 = i - 1
    return i0 // NCOLS, i0 % NCOLS

# Tableau récapitulatif des 40 blocs
rows = []
for r in range(NROWS):
    for c in range(NCOLS):
        rows.append({"id": block_id(r,c), "row": r+1, "col": c+1, "profit":␣
↪PROFITS[r][c]})
df_blocks = pd.DataFrame(rows)
df_blocks.head(10)
```

[1]:

	id	row	col	profit
0	1	1	1	-1
1	2	1	2	-1
2	3	1	3	2
3	4	1	4	1
4	5	1	5	1
5	6	1	6	-1
6	7	1	7	1
7	8	1	8	-2
8	9	2	1	-2
9	10	2	2	-1

1.2 Question 2 — Contraintes de pente 45° (relations de précédence)

La pente 45° impose la règle suivante :

Pour extraire un bloc situé à la ligne (r>1), colonne (c), il faut extraire d'abord les blocs de la ligne au-dessus : - haut-gauche ((r-1, c-1)) - haut ((r-1, c)) - haut-droite ((r-1, c+1))

On construit alors une liste d'arcs (i → j) signifiant : si le bloc **i** est extrait, alors le bloc **j** doit l'être aussi.

[2]:
```
def build_precedence_arcs():
    arcs = []
    for r in range(1, NROWS):      # lignes 2..5 (0-based: 1..4)
        for c in range(NCOLS):
            i = block_id(r, c)
            for dc in (-1, 0, 1):
                c2 = c + dc
                if 0 <= c2 < NCOLS:
                    j = block_id(r-1, c2)
                    arcs.append((i, j))
    return arcs

ARCS = build_precedence_arcs()
print("Nombre d'arcs (contraintes) =", len(ARCS))
print("Exemple d'arcs :", ARCS[:12])
```

```
Nombre d'arcs (contraintes) = 88
Exemple d'arcs : [(9, 1), (9, 2), (10, 1), (10, 2), (10, 3), (11, 2), (11, 3),
(11, 4), (12, 3), (12, 4), (12, 5), (13, 4)]
```

1.3 Question 3 — Modélisation en Programme Linéaire en Nombres Entiers (PLNE)

Variables : [X_i = 1 si le bloc i est extrait, sinon 0]

Objectif :

$$\max Z = \sum_{i=1}^{40} P_i X_i$$

Contraintes de précédence (pente) : pour tout arc ((i →j)), [X_i X_j] et [X_i {0,1}]

[3]:
```
# Ici on ne résout pas encore : on prépare profit(i) pour toute résolution␣
↪(PLNE ou min-cut).
profit = {i: PROFITS[rc_from_id(i)[0]][rc_from_id(i)[1]] for i in range(1,␣
↪NROWS*NCOLS+1)}
profit[1], profit[40]
```

[3]:
```
(-1, -1)
```

1.4 Question 4 — Résolution par Lerchs–Grossmann (min-cut / max-flow)

Le problème de fosse ultime équivaut à un problème de **maximum closure**. On le transforme en graphe avec source (s) et puits (t) :

- Si (P_i 0) : arc (s →i) de capacité (P_i)
- Si (P_i < 0) : arc (i →t) de capacité (|P_i|)
- Pour chaque précédence (i →j) : arc de capacité **très grande** (infini)

La **coupe minimale** donne la solution optimale : les blocs du côté de (s) sont extraits.

[5]:
```
import networkx as nx

def solve_by_min_cut():
    pos_sum = sum(v for row in PROFITS for v in row if v > 0)
    neg_sum = -sum(v for row in PROFITS for v in row if v < 0)
    BIG = pos_sum + neg_sum + 1  # "infini" sûr

    G = nx.DiGraph()
    s, t = "s", "t"
    G.add_node(s); G.add_node(t)

    for r in range(NROWS):
        for c in range(NCOLS):
            i = block_id(r,c)
            w = PROFITS[r][c]
            if w >= 0:
                G.add_edge(s, i, capacity=float(w))
            else:
                G.add_edge(i, t, capacity=float(-w))

    for i, j in ARCS:
        G.add_edge(i, j, capacity=float(BIG))

    cut_value, (S, T) = nx.minimum_cut(G, s, t, capacity="capacity")
```

```
    selected = sorted([n for n in S if n not in (s, t)])
    total_profit = sum(PROFITS[rc_from_id(i)[0]][rc_from_id(i)[1]] for i in␣
 ↪selected)
    return selected, int(total_profit)

selected, profit_opt = solve_by_min_cut()
print("Profit optimal =", profit_opt)
print("Nombre de blocs extraits =", len(selected))
print("Blocs extraits =", selected)
```

```
Profit optimal = 19
Nombre de blocs extraits = 16
Blocs extraits = [1, 2, 3, 4, 5, 6, 7, 10, 11, 12, 13, 14, 19, 20, 21, 28]
```

1.5 Question 5 — Vérification de faisabilité (contraintes de pente)

Une solution est faisable si : pour tout arc ((i →j)), si (i) est extrait alors (j) est extrait.

On teste toutes les contraintes et on liste les violations s'il y en a.

[6]:
```
def check_feasibility(selected):
    sel = set(selected)
    viol = [(i,j) for (i,j) in ARCS if i in sel and j not in sel]
    if viol:
        print("  Solution NON faisable. Violations =", len(viol))
        print("Exemples :", viol[:10])
        raise ValueError("Solution non faisable")
    print("  Solution faisable : toutes les contraintes sont respectées.")

check_feasibility(selected)
```

```
  Solution faisable : toutes les contraintes sont respectées.
```

1.6 Question 6 — Matrice de décision Xi et exports

On construit la matrice (X) (5×8) : - 1 si le bloc est extrait - 0 sinon

Puis on exporte : - `solution_blocks.csv` : liste complète des blocs avec (P_i) et (X_i) - `solution_grid.csv` : matrice 5×8

[7]:
```
import os
import pandas as pd

def build_grid(selected):
    grid = [[0]*NCOLS for _ in range(NROWS)]
    for i in selected:
        r, c = rc_from_id(i)
        grid[r][c] = 1
    return grid
```

```
grid = build_grid(selected)

print("Matrice Xi :")
for row in grid:
    print(" ".join(map(str, row)))

out_dir = "outputs"
os.makedirs(out_dir, exist_ok=True)

# solution_blocks.csv
rows = []
sel = set(selected)
for r in range(NROWS):
    for c in range(NCOLS):
        i = block_id(r,c)
        rows.append({
            "block_id": i,
            "row": r+1,
            "col": c+1,
            "profit": PROFITS[r][c],
            "x": 1 if i in sel else 0
        })
df_sol = pd.DataFrame(rows)
df_sol.to_csv(os.path.join(out_dir, "solution_blocks.csv"), index=False,␣
 ↪encoding="utf-8")

# solution_grid.csv
df_grid = pd.DataFrame(grid, columns=[f"col_{c+1}" for c in range(NCOLS)])
df_grid.to_csv(os.path.join(out_dir, "solution_grid.csv"), index=False,␣
 ↪encoding="utf-8")

print("  Exports créés dans :", out_dir)
```

```
Matrice Xi :
1 1 1 1 1 1 1 0
0 1 1 1 1 1 0 0
0 0 1 1 1 0 0 0
0 0 0 1 0 0 0 0
0 0 0 0 0 0 0 0
  Exports créés dans : outputs
```

1.7 Question 7 — Figures pour le rapport

On génère 3 figures : 1) Carte des profits (avec IDs) 2) Carte Xi (avec IDs) 3) Contour approximatif de la fosse (frontière des blocs extraits)

Fichiers créés dans le dossier `figures/`.

[10]:
```
import numpy as np
import matplotlib.pyplot as plt

P = np.array(PROFITS)
X = np.array(grid)

def plot_map_show(mat, title, annotate_ids=True):
    plt.figure()
    plt.imshow(mat, aspect="auto")
    plt.title(title)
    plt.xlabel("Colonnes")
    plt.ylabel("Lignes")
    plt.xticks(range(NCOLS), [str(c+1) for c in range(NCOLS)])
    plt.yticks(range(NROWS), [str(r+1) for r in range(NROWS)])

    if annotate_ids:
        for r in range(NROWS):
            for c in range(NCOLS):
                plt.text(c, r, str(block_id(r, c)), ha="center", va="center",␣
 ↪fontsize=8)

    plt.tight_layout()
    plt.show()

# 1) Carte des profits
plot_map_show(P, "Carte des profits Pi", annotate_ids=True)

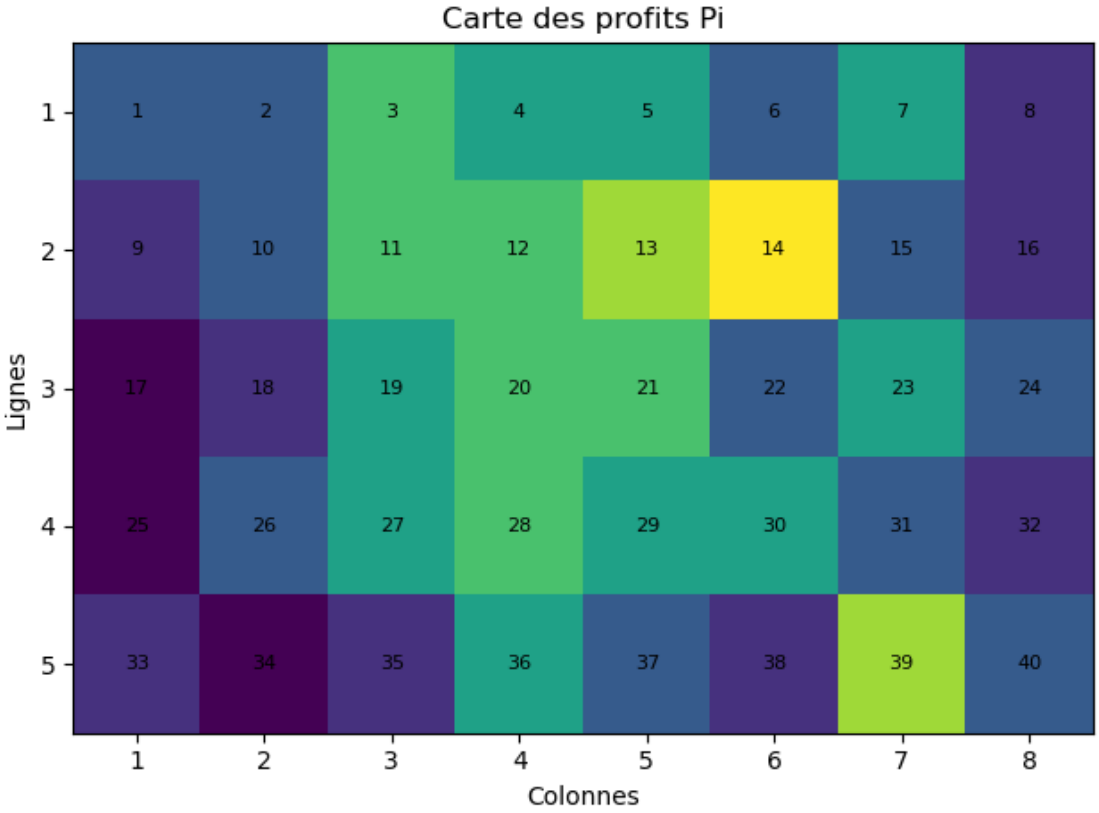
# 2) Carte Xi
plot_map_show(X, "Carte Xi (1=extrait, 0=non)", annotate_ids=True)

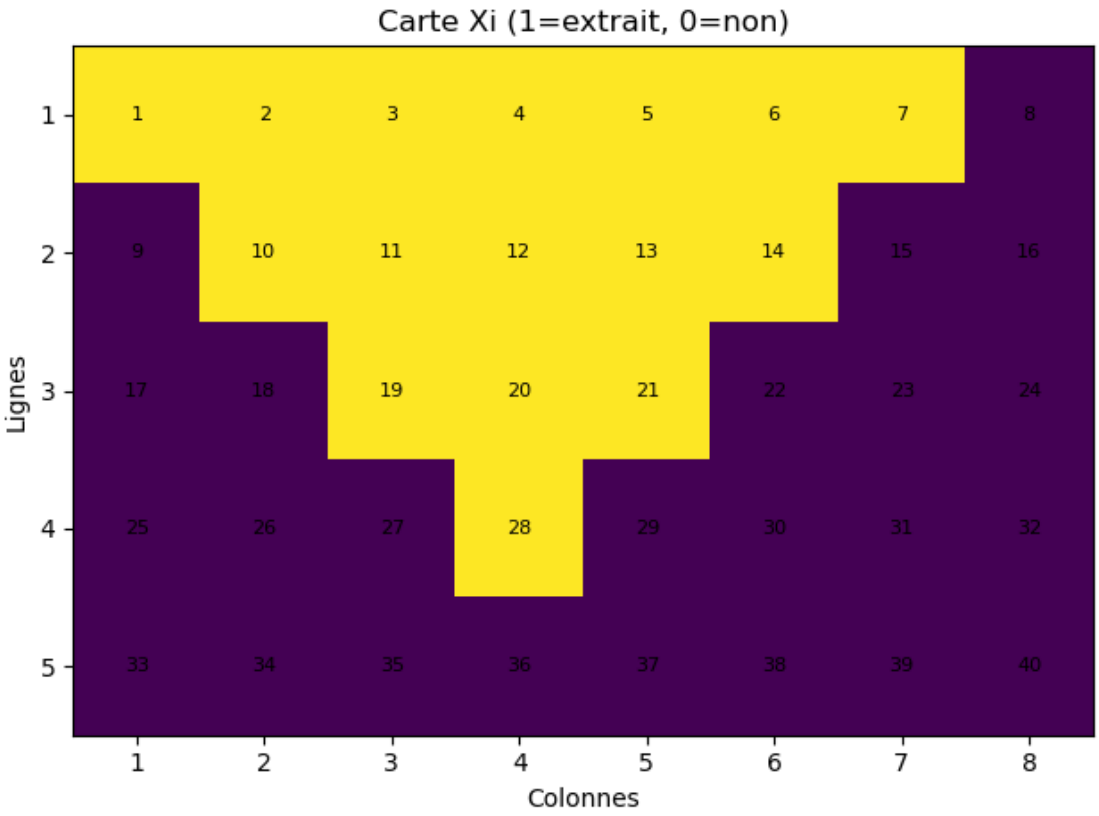
# 3) Contour fosse
plt.figure()
plt.imshow(X, aspect="auto")
plt.title("Contour fosse (frontière des blocs Xi=1)")
plt.xlabel("Colonnes")
plt.ylabel("Lignes")
plt.xticks(range(NCOLS), [str(c+1) for c in range(NCOLS)])
plt.yticks(range(NROWS), [str(r+1) for r in range(NROWS)])

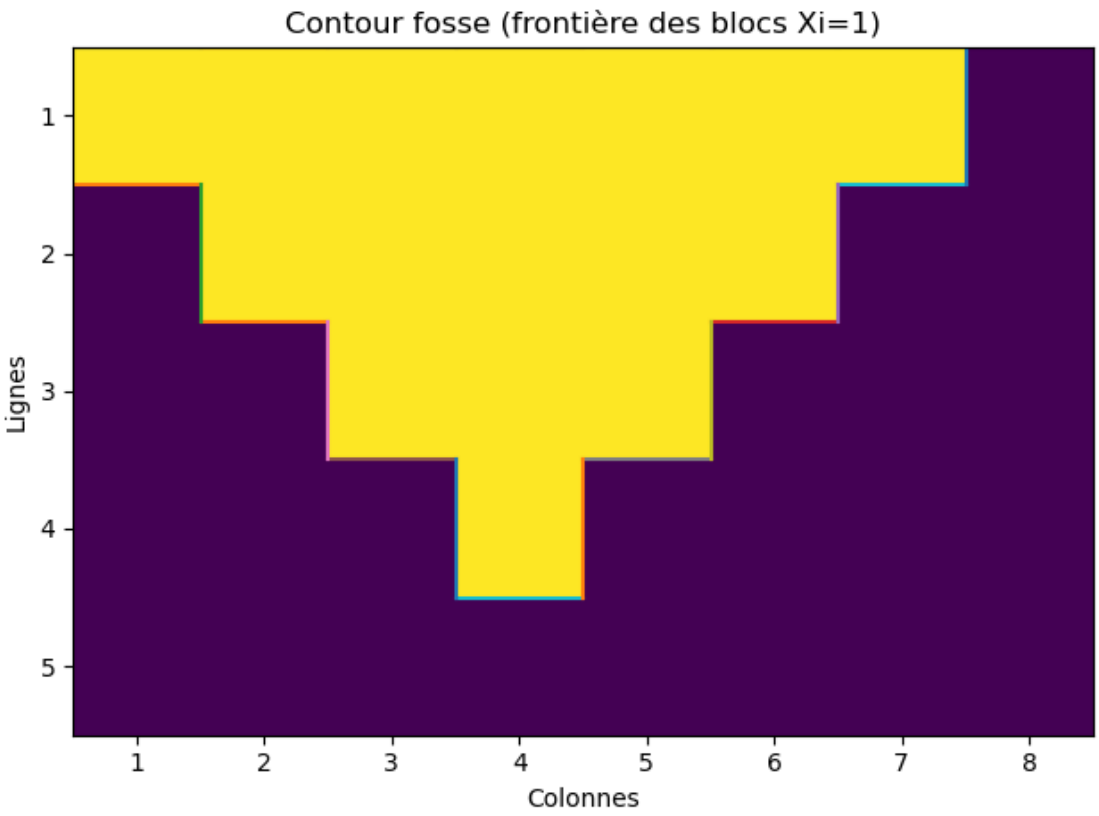
for r in range(NROWS):
    for c in range(NCOLS):
        if X[r, c] == 1:
            if r == 0 or X[r-1, c] == 0:
                plt.plot([c-0.5, c+0.5], [r-0.5, r-0.5])
            if r == NROWS-1 or X[r+1, c] == 0:
                plt.plot([c-0.5, c+0.5], [r+0.5, r+0.5])
            if c == 0 or X[r, c-1] == 0:
```

```
                plt.plot([c-0.5, c-0.5], [r-0.5, r+0.5])
            if c == NCOLS-1 or X[r, c+1] == 0:
                plt.plot([c+0.5, c+0.5], [r-0.5, r+0.5])

plt.tight_layout()
plt.show()
```




Carte Xi (1=extrait, 0=non)
Lignes
Colonnes
1 2 3 4 5 6 7 8
9 10 11 12 13 14 15 16
17 18 19 20 21 22 23 24
25 26 27 28 29 30 31 32
33 34 35 36 37 38 39 40


Contour fosse (frontière des blocs Xi=1)
Lignes
Colonnes
1
2
3
4
5
1
2
3
4
5
6
7
8